一、需求分析

1.1 项目背景

消费者价格指数（CPI）是衡量通胀水平的重要宏观经济指标，但传统CPI依赖人工调研与月度统计更新，存在时效性差、颗粒度粗、覆盖不全等固有缺陷。随着电商平台数据的大规模生成，构建高频（按日）价格指数成为可能。电商数据具备更新快、覆盖面广、精度高等优势，可突破传统CPI的限制，形成对市场更灵敏的监测手段。

1.2 项目目标

1. 构建一个基于电商数据的高频价格指数计算系统
2. 使用阿里云 OSS 进行数据管理
3. 利用 ClickHouse 实现高效查询与价格指数构建
4. 用 Python 自动化计算流程
5. 通过 Quick BI 或 Matplotlib 实现价格指数趋势可视化

二、表结构设计

2.1 `prices` 表（价格记录表）

设计说明

该表记录了电商商品的历史价格快照信息，是价格指数计算的主数据源。通过日期维度可支持日级别高频统计分析。

表设计

字段名	类型	含义
product_id	UInt64	商品 ID，对应 `products` 表主键
category_id	UInt64	商品类别 ID
name	String	商品名称
price	Float32	商品价格
date	Date	价格采集日期

实现代码

```
CREATE TABLE prices (
    product_id UInt64,
    category_id UInt64,
    name String,
    price Float32,
    date Date
) ENGINE = OSS(
  'https://price-index-demo.oss-cn-hangzhou-internal.aliyuncs.com/price.csv',
  'AccessKey',
  'AccessKeySecret',
  'CSVWithNames'
)

```

2.2 `products` 表（商品信息表）

设计说明

该表提供商品的基础信息和统计属性，可用于过滤、排序或权重加权计算。`weight` 字段可支持基于商品重要度的价格指数加权。

表设计

字段名	类型	含义
product_id	UInt64	商品唯一标识
category_id	UInt64	商品所属类别 ID
name	String	商品名称
weight	Float32	商品权重（用于指数加权计算）
price	Float32	商品当前价格
change_count	UInt32	商品历史变价次数

实现代码

```
CREATE TABLE default.products (
    product_id UInt64,
    category_id UInt64,
    name String,
    weight Float32,
    price Float32,
    change_count UInt32
) ENGINE = OSS (
  'https://price-index-demo.oss-cn-hangzhou-internal.aliyuncs.com/products.csv',
  'AccessKey',
  'AccessKeySecret',
  'CSVWithNames'
)
```

```
```

2.3 `categories` 表（商品分类表）

设计说明

该表描述商品的分类结构，支持多层级分类嵌套（如食物 → 粮食 → 大米）。结合 `hierarchy` 和 `parent` 字段可构建完整分类树，用于层级价格指数汇总。`weight` 字段可作为类级别的加权因子，符合 CPI 指数构造逻辑。

表设计

字段名	类型	含义
category	String	类别名称
category_id	UInt64	类别唯一编号（主键）
hierarchy	UInt8	类别所在层级（如 1 表示一级分类）
weight	Float32	类别权重（可用于 CPI 权重加权）
price	Float32	当前该类别的平均价格（可选）
parent	Int64	父类别 ID（如为一级分类则为 -1）

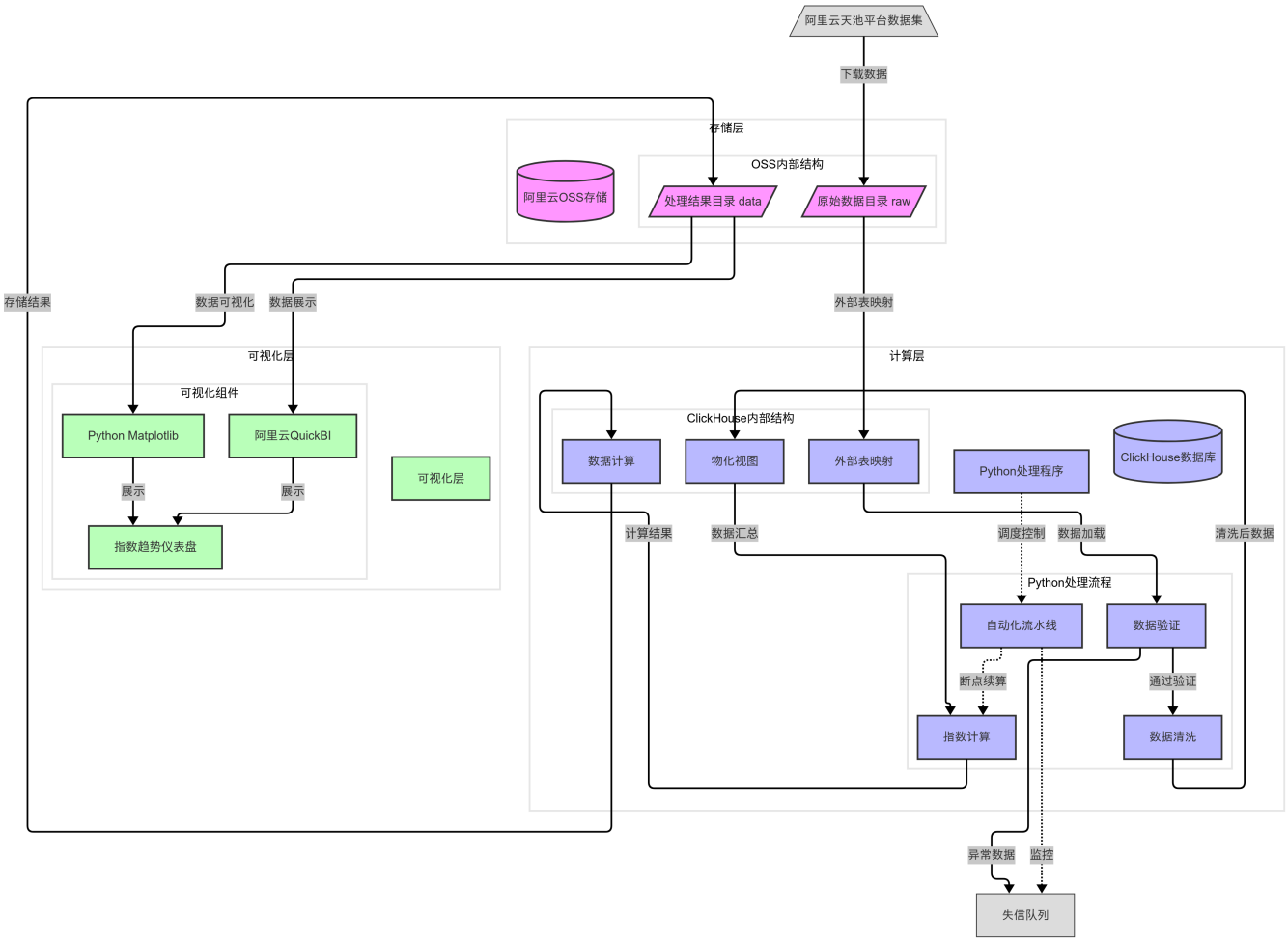
实现代码

```sql
CREATE TABLE default.categories (
    category String,
    category_id UInt64,
    hierarchy UInt8,
    weight Float32,
    price Float32,
    parent Int64
) ENGINE = OSS (
  'https://price-index-demo.oss-cn-hangzhou-internal.aliyuncs.com/categories.csv',
  'AccessKey',
  'AccessKeySecret',
  'CSVWithNames'
)

```

三、系统架构设计

3.1 总体架构设计



3.2 核心Python代码逻辑

3.2.1 数据预处理

3.2.1.1 数据清洗

针对 `daily_price` 文件数据，对其进行数据清洗，其原理主要为：

1. 读取原始产品与类别数据；
2. 遍历 `daily_price` 文件夹下所有 CSV 文件，读取并清洗价格数据（空值或负值等）；
3. 标准化字段与时间格式，删除不合理数据（空值或错误值等）；
4. 合并所有文件并保存为统一格式 `price.csv`；
5. 同步处理 `categories.csv` 文件，填补 null 值为 -1 并保存。

3.2.1.2 编码转换

针对 `price.csv`、`products.csv`、`categories.csv`，由于其保存编码为`GBK`，易出现乱码情况，对其统一转换为`UTF-8-sig`编码

3.2.1.3 上传文件

将处理后的 `price.csv`、`products.csv`、`categories.csv`文件上传至阿里云OSS云存储，并设置阿里云 Click House 与OSS之间的映射关系。

3.2.2 程序初始化

- 加载配置文件
- 通过 `clickhouse-driver` 连接 Click House 数据库
- 初始化 SQLAlchemy 引擎

3.2.3 计算某两个日期之间的CPI

使用 ClickHouse SQL 在数据库中聚合计算：

- 每个产品在基准日和报告日的价格；
- 基于拉氏指数（log 平均比 + exp）计算每个叶子类目的指数；
- 按权重加权求和，得到总体 CPI。

3.2.4 计算每日CPI时间序列

- 加载所有目标日期价格；
- 构建 `product_id` × `date` 的价格透视表（`pivot_table`）；
- 固定起始日价格作为基准，其他日期为报告日；
- 每天对产品做 log(price_ratio) → exp 平均 → 得到类别指数；
- 按类别权重聚合生成每日 CPI；
- 返回 `pd.Series` 类型的 CPI 序列。

3.3 核心SQL代码逻辑

3.3.1 找出叶子类别（即最小子类）

CPI 计算以叶子类别为单位进行加权汇总，因此需要首先提取出不再作为父类别的 category_id。

parent != -1 排除根节点标识，NOT IN 逻辑确保只选取没有子类的末级分类。

每个类别的权重 weight 将用于最终 CPI 加权。

```sql
WITH
leaf_categories AS (
    SELECT category_id, weight
    FROM categories
    WHERE category_id NOT IN (
        SELECT parent FROM categories WHERE parent != -1
    )
),
```

3.3.2 汇总每个产品在基准日期和报告日期的价格

通过 MAXIf 聚合函数筛选出指定日期的价格，提取每个产品在起止两个时间点的价格。

由于电商价格可能存在多条记录，使用 MAXIf 取同一日期下最大值作为该产品的代表价格。

```sql
price_data AS (
    SELECT
        product_id,
        MAXIf(price, toDate(date) = toDate('{start_date}')) AS base_price,
        MAXIf(price, toDate(date) = toDate('{end_date}')) AS report_price
    FROM prices
    WHERE date IN ('{start_date}', '{end_date}')
    GROUP BY product_id
),
```

3.3.3 在每个叶子类别中，计算产品的价格指数（几何平均）

对每个叶子类别，首先将产品按 product_id 聚合到其 category_id。

采用几何平均数来表示价格变动率：

$$\text{价格指数} = \exp(\frac{1}{n}\sum\log(\frac{P_1}{P_0}))$$

利用 log 转换后再 avg 求均值，再 exp 还原，具有对异常值更强的鲁棒性。

```sql
category_cpi AS (
    SELECT
        p.category_id,
        EXP(avg(log(pd.report_price / pd.base_price))) AS price_index
    FROM products p
    JOIN price_data pd ON p.product_id = pd.product_id
    JOIN leaf_categories lc ON p.category_id = lc.category_id
    WHERE pd.base_price > 0 AND pd.report_price IS NOT NULL
    GROUP BY p.category_id
)
```

3.3.4 聚合得到加权CPI值

将每个叶子类别的价格指数 `price_index` 按其对应的权重 `weight` 加权汇总，最终返回单一 CPI 指数值。

```sql
SELECT
    SUM(cc.price_index * lc.weight) AS CPI
FROM category_cpi cc
JOIN leaf_categories lc ON cc.category_id = lc.category_id;
```

四、测试与验证设计

一、目的

为了确保商品价格数据清洗脚本的正确性、鲁棒性与兼容性，对各模块执行功能测试和边界测试，验证其在不同输入条件下的稳定性与准确性，确保输出数据格式统一、字段规范、内容完整，为后续高频价格指数计算提供可靠的数据基础。

二、测试内容与方法

本次测试设计包括**单元测试**与**集成测试**两个层级，分别验证模块功能实现与系统整体表现。

1）单元测试

采用构造式测试方法，手工创建多个小规模 CSV 文件，覆盖常见与极端场景，重点测试以下模块：

- **字段提取模块**：验证脚本能否准确识别并解析 `product_id`、`category_id`、`price`、`change_date` 等字段；
- **日期解析模块**：测试多种日期格式（如 `yyyy/MM/dd`、`dd-MM-yyyy` 等）是否能被统一转换为 `yyyy-MM-dd` 格式；
- **价格过滤模块**：验证价格为负值、缺失或异常字符（如“NaN”、“--”）的记录能否被正确识别并剔除；
- **输出模块**：检查输出文件编码格式是否为 `utf-8-sig`，字段顺序是否正确，文件内容是否完整；
- **兼容性测试**：测试脚本在不同系统环境（Windows/Linux）下的路径解析与读写兼容性。

测试均通过，说明各模块功能稳定、逻辑清晰，具备良好的可维护性。

2）集成测试

在完成各模块单元测试后，使用实际采集数据执行整体流程测试，验证系统在真实数据规模下的处理能力与准确性，包含以下内容：

- **商品覆盖率测试**：统计清洗后数据中商品种类数与原始商品列表对比，计算覆盖率；
- **异常数据识别率测试**：使用标注样本比对被剔除数据与预期异常数据集，计算准确率。
- **数据一致性测试**：清洗结果导入 ClickHouse 后进行字段查询，验证字段类型与值域是否与预期一致，确保后续分析环节无格式障碍。